Questionário (ref. Aulas 01 a 08)

Estrutura e Configuração das Camadas

1) **O que é um neurônio artificial?**
   É a unidade básica de processamento de uma rede neural. Ele recebe valores de entrada, realiza uma **combinação linear dessas entradas utilizando pesos e bias** e, em seguida, aplica uma **função de ativação** para produzir sua saída.

2) **Qual é a função da camada de entrada?**
   É a **interface da rede com os dados de entrada**. Recebe os atributos em formato numérico (como valores de dados tabulares, pixels de imagens etc.) e os encaminha para a **primeira camada oculta**. Em uma MLP tradicional, a camada de entrada **não possui pesos, bias nem função de ativação**.

3) **O que define a quantidade de elementos (ou "nós") na camada de entrada?**
   A camada de entrada possui **um elemento para cada atributo de entrada**. Portanto, sua quantidade é determinada pela dimensionalidade dos dados. Se cada instância possui 15 valores de entrada, a camada terá 15 nós; se, por outro lado, possuir 500 valores, terá 500 nós.

4) **A camada de entrada realiza algum processamento matemático?**
   Não. Em uma MLP tradicional, a camada de entrada apenas recebe os valores dos atributos e os encaminha para a primeira camada oculta. Ela funciona como uma "interface da rede neural com o mundo externo", de maneira análoga aos sentidos no sistema biológico. Ela não possui pesos, *bias* nem função de ativação.

5) **O que define a quantidade de neurônios nas camadas ocultas?**
   A quantidade de neurônios nas camadas ocultas é um **hiperparâmetro de arquitetura**, definido no projeto da rede e normalmente ajustado experimentalmente. Uma entrada com 20 valores pode alimentar uma camada com 32, 64 ou 128 neurônios; da mesma forma, uma entrada com 500 valores pode alimentar uma camada com apenas 64 neurônios. A escolha influencia a **capacidade de representação e o custo computacional** da rede.

6) **Existe uma regra para definir a quantidade de neurônios nas camadas ocultas?**
   Não existe uma regra universal. A quantidade de neurônios é uma **hipótese de arquitetura que deve ser validada experimentalmente**. Na prática, são comuns valores como **32, 64, 128 e 256**, por convenção e eficiência computacional. A configuração mais adequada deve ser determinada durante o *tuning*, testando diferentes combinações e avaliando o desempenho no conjunto de validação.

7) **O que determina a quantidade de neurônios da camada de saída?**
   A quantidade de neurônios da camada de saída é determinada pelo **tipo de problema e pela forma como a saída deve ser representada**. Em uma regressão com uma única variável-alvo, normalmente utiliza-se **1 neurônio**; em classificação binária, geralmente **1 neurônio**; e, em classificação multiclasse ou multilabel, normalmente utiliza-se **1 neurônio por classe**.

8) **A camada de entrada é considerada na profundidade da rede?**
   Depende da convenção adotada. Geralmente, a camada de entrada **não é contabilizada na profundidade**, pois apenas recebe e encaminha os dados, sem realizar transformação computacional. Assim, uma rede com **uma camada de entrada, duas camadas ocultas e uma camada de saída** pode ser descrita como uma rede com **três camadas computacionais**, embora possua quatro camadas se a entrada também for contabilizada.

Perceptron

9) **O que é o Perceptron?**
   O Perceptron é um dos primeiros modelos de neurônio artificial com capacidade de aprendizado. Em sua forma simples, é utilizado em **classificação binária** e aprende uma **fronteira de decisão linear**

10) **Qual é a principal limitação do Perceptron?**
    O Perceptron simples consegue resolver apenas problemas **linearmente separáveis**. Sua limitação clássica é o problema **XOR**, cujas classes não podem ser separadas corretamente por uma única reta.

Processamento nas Camadas

11) **Qual é o papel das camadas ocultas?**
    As camadas ocultas permitem que a rede construa representações intermediárias e aprenda relações complexas presentes nos dados.

12) **O que um neurônio de uma camada oculta faz?**
    Ele recebe as saídas da camada anterior, calcula uma soma ponderada com pesos e *bias* e aplica uma função de ativação para produzir sua saída.

13) **Qual é o papel da camada de saída?**
    Transformar as representações internas produzidas pela rede em uma **resposta adequada ao tipo de problema e ao formato esperado da saída**.

14) **A camada de saída também possui pesos e bias?**
    Sim. Os pesos e *biases* da camada de saída também são parâmetros aprendidos durante o treinamento.

15) **Uma rede neural possui quatro camadas: entrada com 4 nós, duas camadas ocultas com 4 e 2 neurônios e uma camada de saída com 4 neurônios. Quantos parâmetros treináveis ela possui?**
    Em cada camada, o número de parâmetros é calculado **multiplicando a quantidade de entradas recebidas pelo número de neurônios** da camada e **somando 1 bias para cada neurônio**. A primeira camada oculta possui $(4 \times 4) + 4 = 20$ parâmetros. A segunda possui $(4 \times 2) + 2 = 10$ parâmetros. A camada de saída possui $(2 \times 4) + 4 = 12$ parâmetros. Portanto, a rede possui 42 parâmetros treináveis.

16) Qual é a função do bias em um neurônio artificial?
O bias é um parâmetro aprendido durante o treinamento que é somado à combinação ponderada das entradas (pré-ativação ou, simplesmente, "z"). Ele permite deslocar a função do neurônio, dando maior flexibilidade ao modelo e **evitando que a fronteira de decisão fique necessariamente presa à origem**.

Funções de Ativação

17) Como são escolhidas as funções de ativação?
Nas **camadas ocultas**, a função de ativação é uma escolha de arquitetura e pode ser tratada como **hiperparâmetro** durante o *tuning*. Na **camada de saída**, sua escolha depende do **tipo de problema e da forma desejada para a saída**.

18) Por que as funções de ativação são importantes nas camadas ocultas?
Porque introduzem **não linearidade** na rede. Sem elas, sucessivas camadas lineares equivaleriam a uma única transformação linear, limitando a capacidade da rede de aprender relações complexas.

19) O papel da função de ativação na camada de saída é o mesmo papel que elas ocupam nas camadas ocultas?
Não. Nas camadas ocultas, sua principal função é introduzir **não linearidade**. Na camada de saída, sua principal função é **adequar a resposta ao tipo de problema**, como produzir uma probabilidade ou uma distribuição de probabilidades.

Hiperparâmetros

20) Qual é a diferença entre parâmetros e hiperparâmetros em uma rede neural?
Os **parâmetros** são aprendidos pela rede durante o treinamento, como os **pesos e biases**. Os **hiperparâmetros** são definidos pelo analista antes do treinamento e podem ser ajustados durante o processo de *tuning*, determinando **como a rede será estruturada e como o aprendizado ocorrerá**.

21) Dê exemplos de parâmetros e hiperparâmetros de uma rede neural.
Exemplos de **parâmetros**: pesos e biases. Exemplos de **hiperparâmetros**: número de camadas ocultas, número de neurônios por camada, função de ativação, tamanho do lote, número de épocas, função de perda, otimizador e taxa de aprendizado. Em outras palavras, os **parâmetros** representam **o que** a rede aprende, enquanto os **hiperparâmetros** definem **como** esse aprendizado ocorrerá.

Hiperparâmetros de Arquitetura

22) Quais são os principais hiperparâmetros de arquitetura de uma rede neural?
São principalmente o **número de camadas ocultas**, o **número de neurônios por camada** e a **função de ativação** utilizada.

23) Existe uma regra para definir o número de camadas ocultas de uma RN?

Não. Essa escolha depende da complexidade do problema e deve ser validada experimentalmente. Em problemas tabulares tradicionais, **1 a 3 camadas ocultas** costumam ser um ponto de partida razoável.

24) **Existe uma regra para definir o número de neurônios em cada camada oculta?**
Não. Essa quantidade também é definida experimentalmente. Valores como **32, 64, 128 e 256** são comuns, e mais neurônios aumentam a capacidade de representação, mas também o custo computacional.

25) **O que representam a profundidade e a largura de uma rede neural?**
A **profundidade** está relacionada à quantidade de camadas ocultas, enquanto a **largura** corresponde à quantidade de neurônios em cada camada. Ambas são definidas como hiperparâmetros da arquitetura.

26) **Qual é a diferença entre o papel das funções de ativação nas camadas ocultas e na camada de saída?**
Nas camadas ocultas, sua principal função é introduzir **não linearidade**. Na camada de saída, sua função é adequar a resposta da rede ao **tipo de problema e ao formato esperado da saída**.

27) **Quais são as principais características da função Sigmoid?**
A Sigmoid produz valores entre **0 e 1**, possui formato em "S" e é centrada em 0,5. A partir de aproximadamente **z = ±2,2**, sua saída já se aproxima de 0 ou 1 e a função começa a entrar em uma região de saturação, podendo contribuir para o problema de **vanishing gradient**.

28) **Quais são as principais características da função Tanh?**
A Tangente Hiperbólica produz valores entre **-1 e 1**, possui formato em "S" e é centrada em zero. A partir de aproximadamente **z = ±1,47**, sua saída já se aproxima de -1 ou 1 e a função começa a entrar em uma região de saturação, podendo também sofrer com **vanishing gradient**.

29) **O que é o vanishing gradient (gradiente que desaparece)?**
É o problema em que, durante a retropropagação, os gradientes vão ficando **muito próximos de zero**. Com isso, as atualizações dos pesos ficam muito pequenas e as camadas anteriores da rede passam a **aprender muito lentamente ou praticamente deixam de aprender**.

30) **Quais são as principais características da função ReLU?**
A ReLU produz **0 para valores negativos ou iguais a zero** e mantém o próprio valor quando ele é positivo. É simples, eficiente e reduz o problema de vanishing gradient na região positiva, embora possa apresentar o problema de **dying ReLU**.

31) **O que é o dying ReLU e como pode ser reduzido?**
O **dying ReLU** ocorre quando um neurônio passa a receber valores negativos e a ReLU produz **sempre 0**. Como nessa região o gradiente também é **0**, os pesos deixam de ser atualizados e o neurônio pode parar de aprender. Uma forma comum de reduzir esse problema é usar variantes como a **Leaky ReLU**, que mantém uma pequena inclinação constante para valores negativos.

Questionário (ref. Aulas 01 a 08)

Hiperparâmetros de Treinamento

32) Quais são os principais hiperparâmetros de treinamento de uma RN?
São o **tamanho do lote (batch size)**, o **número de épocas (epochs)**, a **função de perda (loss function)**, o **otimizador** e a **taxa de aprendizado (learning rate)**.

33) O que é o tamanho do lote (batch size)?
É a quantidade de instâncias processadas antes de cada atualização dos pesos e biases. Um **batch maior** utiliza mais amostras para calcular o gradiente, produzindo uma estimativa **mais estável**, mas realiza **menos atualizações** por época. Um **batch menor** utiliza menos amostras, produzindo uma estimativa **menos estável**, mas realiza **mais atualizações** por época.

34) O que é uma época (epoch)?
É uma passagem completa de **todo o conjunto de treinamento pela rede**. Durante uma época, os dados são processados em lotes e os parâmetros são atualizados à medida que esses lotes são processados. **Poucas épocas** podem favorecer **underfitting** (a rede não aprende os padrões dos dados); **muitas épocas** podem favorecer **overfitting** (a rede passa a se ajustar excessivamente aos dados de treinamento, reduzindo sua capacidade de generalização).

35) Qual é a relação entre batch size, número de épocas e quantidade de atualizações dos parâmetros?
O batch size determina quantas instâncias são processadas antes de cada atualização. Assim, quanto menor o lote, maior tende a ser o número de atualizações por época. O número de épocas determina quantas vezes todo o dataset será percorrido.

36) Qual é a função da função de perda (loss function)?
Medir o erro do modelo comparando a saída prevista com o valor esperado. Durante o treinamento, a rede ajusta seus parâmetros buscando minimizar essa perda.

37) A mesma função de perda é utilizada para todos os tipos de problema?
Não. A função de perda depende do problema. Em regressão, por exemplo, podem ser utilizadas MSE ou MAE; em classificação, funções como Binary Cross-Entropy ou Categorical Cross-Entropy.

38) O que é a taxa de aprendizado (learning rate)?
É o hiperparâmetro que controla a **magnitude das alterações realizadas nos pesos e biases** a cada atualização. Taxas muito altas podem dificultar a convergência, enquanto taxas muito baixas podem tornar o aprendizado lento.

39) O que é o gradiente e qual é sua função no treinamento?
O gradiente indica, para cada peso e bias, **como e quanto a perda varia em relação àquele parâmetro**, ou seja, sua **direção e intensidade de influência sobre a perda**. Seu sinal indica a direção local de aumento da perda; para minimizá-la, os parâmetros são atualizados na direção oposta.
Obs.: O gradiente **não é um hiperparâmetro**!

40) Qual é a função do otimizador?

O otimizador é o algoritmo responsável por **definir como os gradientes, depois de calculados na backpropagation, serão utilizados para atualizar os pesos e biases** da rede. Ele determina a estratégia de atualização dos parâmetros, buscando reduzir a função de perda ao longo do treinamento. Diferentes otimizadores, como **SGD, RMSprop e Adam**, utilizam os gradientes de maneiras distintas, podendo alterar a velocidade e a estabilidade da convergência.

41) O que é o SGD (Stochastic Gradient Descent)?

O **SGD** é um otimizador que atualiza cada peso e bias diretamente a partir de seu **gradiente** e da **taxa de aprendizado**. No SGD puro, **o gradiente é multiplicado pela taxa de aprendizado** para determinar o tamanho da alteração, e o parâmetro é atualizado na direção oposta ao gradiente, buscando reduzir a função de perda. É um método simples, mas pode apresentar oscilações durante o treinamento. **É uma das formas mais básicas e tradicionais de otimização por gradiente, servindo de base para métodos mais sofisticados.**

42) O que é o Adam?

O **Adam** é um otimizador que também utiliza os gradientes, mas considera **informações acumuladas sobre sua direção e suas magnitudes recentes**. Ele combina ideias de otimizadores anteriores, como **Momentum** e **RMSprop**, tornando as atualizações dos parâmetros mais adaptativas e, em muitos casos, favorecendo uma convergência mais rápida e estável. É atualmente **um dos otimizadores mais utilizados**, especialmente em redes profundas, sendo frequentemente adotado como uma escolha inicial quando não há motivo específico para utilizar outro otimizador.

CNN - Redes Neurais Convolucionais

43) O que é uma Rede Neural Convolucional (CNN)?

É uma arquitetura de rede neural especialmente adequada para dados que possuem **estrutura espacial**, como imagens. Em vez de conectar cada neurônio a toda a entrada, utiliza **filtros que percorrem pequenas regiões** da representação dos dados para identificar padrões locais.

44) O que é um filtro (kernel) em uma camada convolucional?

É um conjunto de **pesos** que percorre a entrada realizando operações de convolução. Diferentes filtros podem aprender a detectar diferentes características, como bordas, texturas, formas e padrões mais complexos.

45) O que representam kernel_size e filters em uma camada Conv2D?

O parâmetro **kernel_size** define as dimensões espaciais do filtro, como 3x3 ou 5x5. Já o parâmetro **filters** define **quantos filtros diferentes** a camada possui e, consequentemente, **quantos mapas de características ela produzirá**.

Questionário (ref. Aulas 01 a 08)

46) O que é o stride?
É o **passo** utilizado pelo filtro ao percorrer a entrada. Com **stride = 1**, ele avança uma posição por vez; com **stride = 2**, avança duas posições por vez. **Strides maiores** normalmente **reduzem as dimensões espaciais da saída**.

47) O que é padding e qual sua finalidade?
É a **adição de valores**, normalmente zeros, ao redor da entrada. Ele permite que as regiões próximas às bordas também sejam processadas de maneira mais adequada. Também permite controlar as dimensões espaciais da saída.

48) Se uma imagem RGB possui três canais, quantas matrizes formam a entrada?
Três matrizes: uma representando o canal vermelho (R), uma o verde (G) e uma o azul (B). Elas são tratadas conjuntamente como uma única entrada com profundidade 3.

49) O que é a profundidade de um filtro convolucional?
É a **quantidade de canais sobre os quais o filtro atua**. A profundidade do filtro é determinada **automaticamente** pela profundidade da entrada. Portanto, se a entrada possui 3 canais e o kernel_size é 3 x 3, cada filtro terá dimensões 3 x 3 x 3.

50) Um filtro aplicado a uma entrada RGB gera três mapas de características?
Não. O filtro possui uma matriz de pesos para cada canal da entrada. Os resultados obtidos nos diferentes canais são somados, juntamente com o bias, produzindo **uma única matriz** Z. Após a função de ativação, obtém-se **um único mapa de características** A. Que define a quantidade de mapas de características (matriz A) é a quantidade de filtros.

51) Se uma camada possui 8 filtros, quantos mapas de características ela produz?
Produz 8 mapas de características, pois **cada filtro produz um mapa**. Esses mapas formam a saída da camada, que terá profundidade 8.

52) Se a saída de uma camada possui 8 mapas de características, qual será a profundidade dos filtros da próxima camada?
Cada filtro da próxima camada terá profundidade 8, pois cada filtro deve atuar sobre **todos os canais/mapas recebidos da camada anterior**.

53) Considere Conv2D (filters=16, kernel_size=(5,5), strides=(2,2)) recebendo uma entrada com 8 canais. Quantos filtros existem e qual é a dimensão de cada um?
Existem 16 filtros. Cada filtro possui dimensão: 5 x 5 x 8. Portanto, cada filtro contém 8 matrizes 5 x 5, uma para cada canal da entrada, e a camada produzirá 16 mapas de características. Essa conta vai cair!

54) Os diferentes filtros de uma mesma camada recebem entradas diferentes?
Não. Todos recebem a **mesma entrada**, mas possuem pesos diferentes. Por isso, podem aprender características diferentes e produzir mapas de características diferentes. Quanto mais filtros, maior a capacidade de aprendizado da rede.

55) Qual é o papel da função de ativação em uma camada convolucional?
Assim como nas camadas ocultas de uma MLP, ela introduz não linearidade.

Questionário (ref. Aulas 01 a 08)

56) Como o tamanho do kernel, o stride e o padding influenciam as dimensões espaciais do mapa de características?

O tamanho do kernel, o stride e o padding determinam quantas posições o filtro poderá assumir como "válidas" ao longo do processamento da entrada. Kernels maiores e strides maiores tendem a reduzir as dimensões espaciais da saída. O padding pode ser utilizado para controlar essa redução. Por exemplo, uma entrada 28 × 28 com kernel 3 × 3, stride 1 e sem padding produz uma saída 26 × 26; com padding adequado (*same*), pode manter 28 × 28.

57) Como é calculada a quantidade de parâmetros treináveis de uma camada convolucional?

Cada filtro possui kernel_height × kernel_width × canais_de_entrada pesos e um bias. Assim:

$$\hat{\text{P}}\text{arametros} = (K_h \times K_w \times C_{in} + 1) \times C_{out}$$

Por exemplo, uma camada com 16 filtros 3 × 3 recebendo uma imagem RGB possui:

$$(3 \times 3 \times 3 + 1) \times 16 = 448$$

parâmetros treináveis.

58) Por que uma CNN normalmente utiliza muito menos parâmetros que uma camada densa ao processar uma imagem?

Porque os mesmos pesos de um filtro são reutilizados enquanto ele percorre diferentes regiões da entrada. Portanto, a quantidade de parâmetros da camada convolucional depende do tamanho dos filtros, da quantidade de canais de entrada e da quantidade de filtros, e não diretamente da altura e largura da imagem.